158.337
Database Development

Week 7

Procedural SQL

(Procedures, Functions & Data types)

Objectives

Understanding stored procedures –

- PL/SQL,
- Functions,
- Scalar data types,
- Anchored data types
- Exception Handling.

Procedural SQL

SQL does not support conditional execution

Isolate critical code

- All applications access shared code
- Better maintenance and logic control

Persistent stored module (PSM) is a block of code containing:

- Standard SQL statements
- Procedural extensions
- Stored and executed at the DBMS server

Procedural SQL (cont'd.)

Procedural SQL enables you to:

- Store procedural code and SQL statements in database
- Merge SQL and traditional programming constructs

Procedural code executed by DBMS when invoked by end user

- Anonymous PL/SQL blocks and triggers
- Stored procedures and PL/SQL functions

PL/SQL

PL/SQL is a proprietary language of Oracle. A PL/SQL processing engine is built into the Oracle database, in which PL/SQL program units can be stored in the database and be called from other development environments (Java, Visual Basic).

PL/SQL is a procedural language that is used to provide logical processing capabilities in an application.

In a three-tier model – user interface, program logic and database - the role of PL/SQL is to provide the program logic.

How PL/SQL gets executed

PL/SQL is not a stand-alone programming language. It exists only as a tool within the Oracle programming environment.

There are two types of PL/SQL blocks:

1. Anonymous PL/SQL Blocks – do not have names, so they cannot be stored in the database and be referenced later. It is compiled each time it is sent to the server.

2. Named PL/SQL Blocks – are stored in the database and can be referenced later by their names. They are used when creating subroutines (i.e. procedures, functions and packages).

Anonymous PL/SQL Block Structure

DECLARE

Definition of identifiers e.g. variables, constants, cursors

-- Optional section

BEGIN

Here you write statements to manipulate variables in declaration section.

-- mandatory section

EXCEPTION

Here you resolve run-time errors

-- Optional section

END;

```
DECLARE
    --Declaration Statements
    variable_name data_type_declaration;

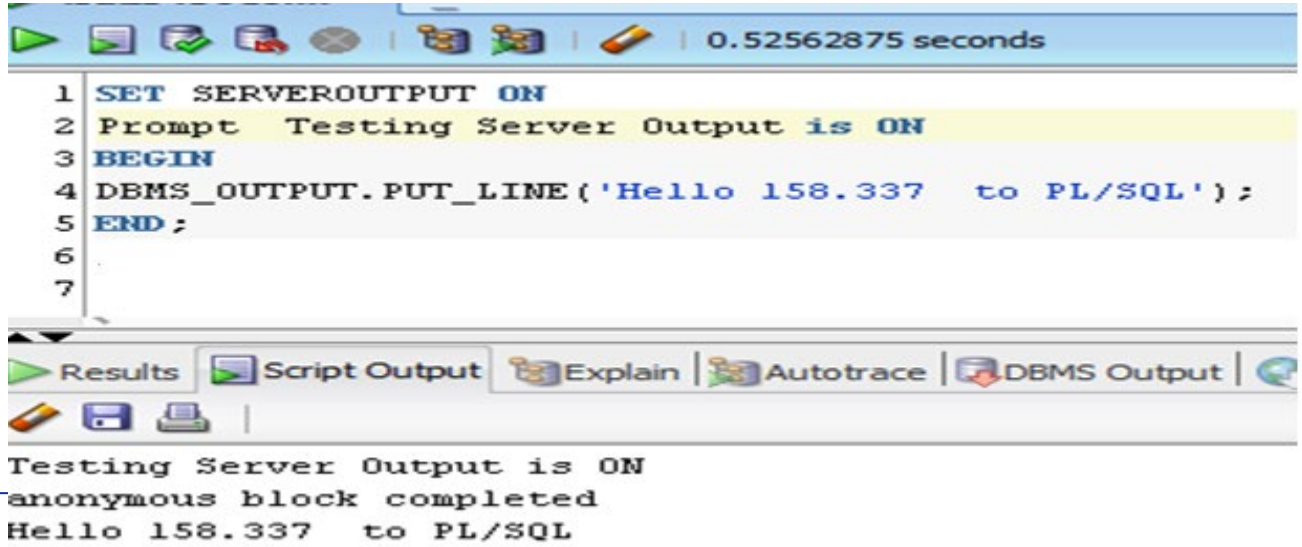
BEGIN
    -- Executable Statements
      variable_name := value;
    DBMS_OUTPUT.PUT_LINE('display_text');

EXCEPTION
  -- Exception-handling Statements
END;
```

SET SERVEROUTPUT ON

The PL/SQL block is stored and executed at the DBMS server. To display the output from the server, we need to explicitly write this statement - SET SERVEROUTPUT ON - at the start of the user session

```
SET SERVEROUTPUT ON
Prompt  Testing Server Output is ON
BEGIN
DBMS_OUTPUT.PUT_LINE('Hello 158.337  to PL/SQL');
END;
```



Anonymous PL/SQL Block

A simple PL/SQL Block to demonstrate use of 3 sections

- Declare
- Begin and
- End

DBMS_OUTPUT.PUT_LINE will always display the server output in a string. So, we will use this command always to verify if our statement runs properly

The assignment symbol := is used to initialize a variable to a value

```
1  DECLARE
2       v_test_date DATE := sysdate;
3      v_test_num NUMBER(3) := 10;
4       v_test_txt VARCHAR2(10);
5  BEGIN
6       v_test_txt :=  '???????';
7      DBMS_OUTPUT.PUT_LINE(v_test_date);
8      DBMS_OUTPUT.PUT_LINE(v_test_num);
9      DBMS_OUTPUT.PUT_LINE(v_test_txt);
10 END;
```

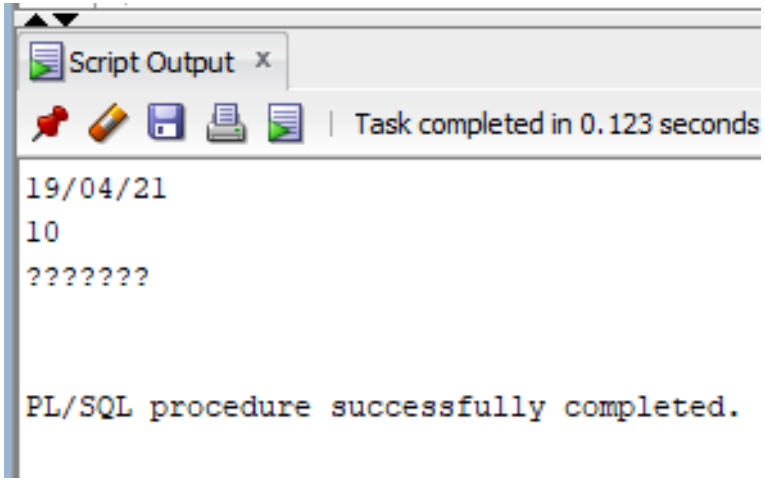


TABLE 8.8

PL/SQL Basic Data Types

DATA TYPE	DESCRIPTION
CHAR	Character values of a fixed length; for example: W_ZIP CHAR(5)
VARCHAR2	Variable-length character values; for example: W_FNAME VARCHAR2(15)
NUMBER	Numeric values; for example: W_PRICE NUMBER(6,2)
DATE	Date values; for example: W_EMP_DOB DATE
%TYPE	Inherits the data type from a variable that you declared previously or from an attribute of a database table; for example: W_PRICE PRODUCT.P_PRICE%TYPE Assigns W_PRICE the same data type as the P_PRICE column in the PRODUCT table

And also BOOLEAN is used in PL/SQL. The three permissible values for BOOLEAN are TRUE, FALSE and NULL.

Scalar Variable Types

Scalar data types hold a single value of types as:

Character – CHAR(n)
VARCHAR2(n)

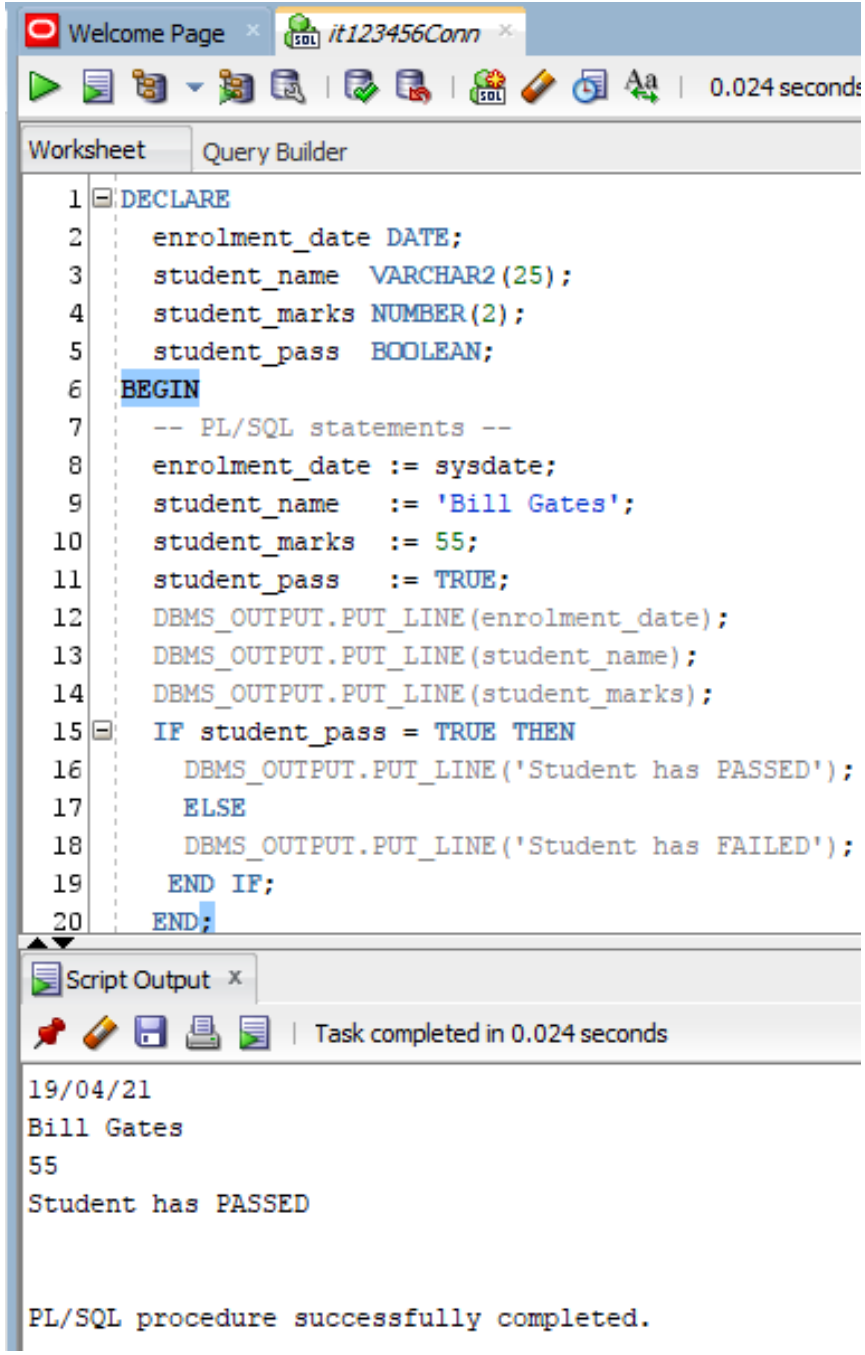
Numeric – NUMBER(p,s)
BINARY_INTEGER

Date – DATE

Boolean – BOOLEAN

The data types for scalar variables correspond to Oracle column data types.

set verify off;
show errors



Concatenate: Joining two separate strings

```
new_string := string1 || string2;
```

LTRIM function: Remove blank leading spaces

```
string := LTRIM(string_variable_name);
```

RTRIM function: Remove blank trailing spaces

```
string := RTRIM(string_variable_name);
```

LENGTH function:

```
string_length := LENGTH(string_variable_name);
```

Character String :

```
string := UPPER(string_variable_name);
string := LOWER(string_variable_name);
string := INITCAP(string_variable_name);
```

INSTR function: Searches string for specific substring

```
start_position := INSTR(original_string, substring);
```

SUBSTR function: Extracts specific number of characters from string

```
extracted_string := SUBSTR(string_variable,starting_point, number_of_characters);
```

Named PL/SQL Procedure Structure

```
CREATE [OR REPLACE] PROCEDURE <procedure_name>
    [(parameter_list)] AS
<declarations>
    Definition of identifiers e.g. variables, constants, cursors
BEGIN
    Here you write statements to manipulate variables in
    declaration section. This section is mandatory
EXCEPTION
    Here you resolve run-time errors
END;
.
/
EXECUTE procedure_name (parameter_list);
```

PL/SQL Blocks

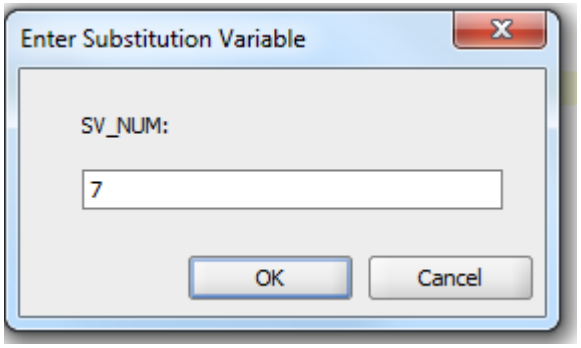
Named PL/ SQL statements

Advantages

- Substantially reduce network traffic and increase performance
- No transmission of individual SQL statements over network
- Reduce code duplication by means of code isolation and code sharing and reuse
- Minimize chance of errors and cost of application development and maintenance

Anonymous PL/SQL Block

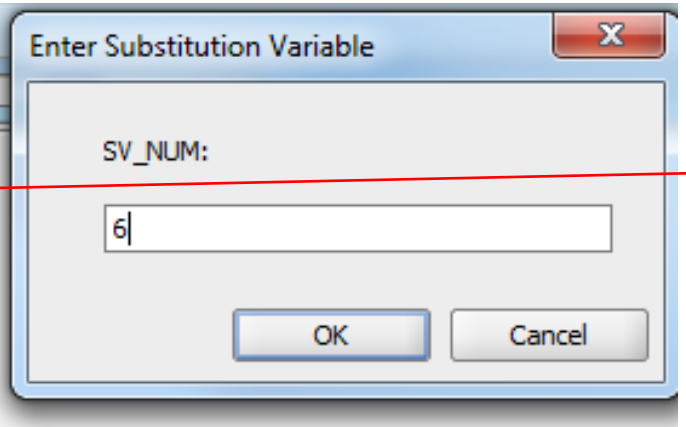
```
1 declare
2 v_num number := &sv_num;
3 v_result number;
4 begin
5 v_result := power(v_num, 2);
6 dbms_output.put_line(' The value of v_result is ' || v_result);
7 end;
```

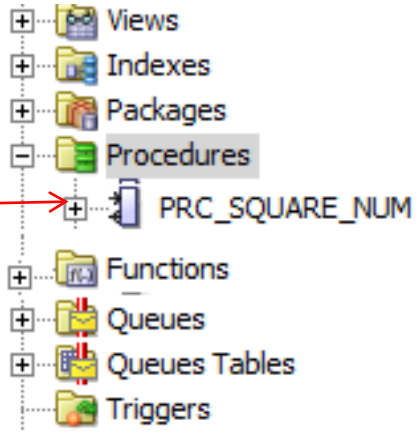



```
anonymous block completed
 The value of v_result is 49
```

Named PL/SQL Block

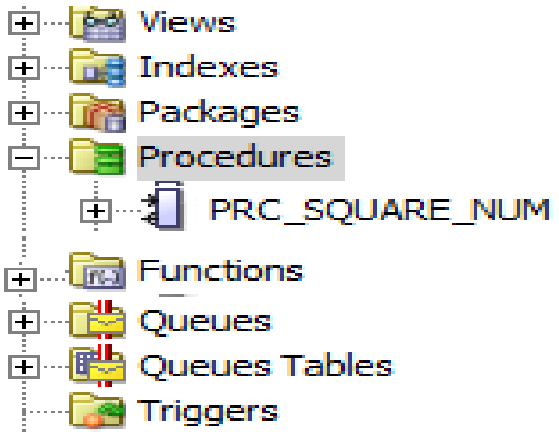
```
1  CREATE OR REPLACE
2  PROCEDURE prc_square_num
3  AS
4    v_num    NUMBER := &sv_num;
5    v_result NUMBER;
6  BEGIN
7    v_result := power(v_num, 2);
8    dbms_output.put_line(' The value of v_result is ' || v_result);
9  END;
```





Named PL/SQL Blocks

AS you can see, named PL/SQL Blocks are stored in the user's procedure section



To use these procedures, we have to explicitly call them

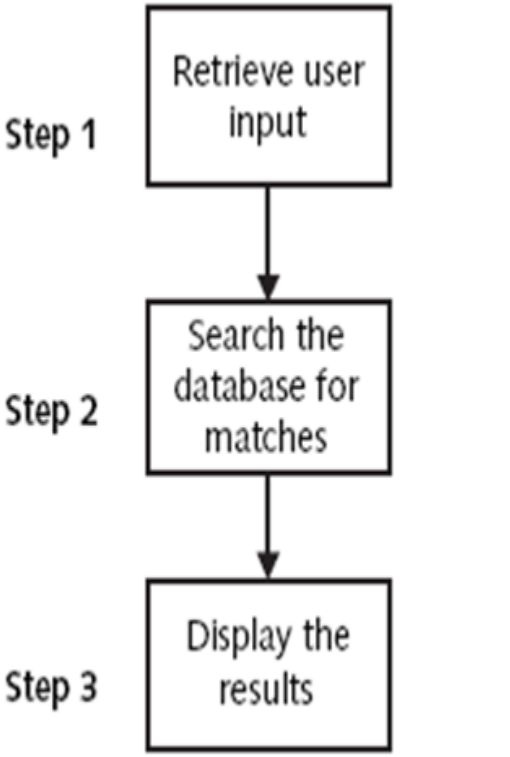


Named procedures can be dropped

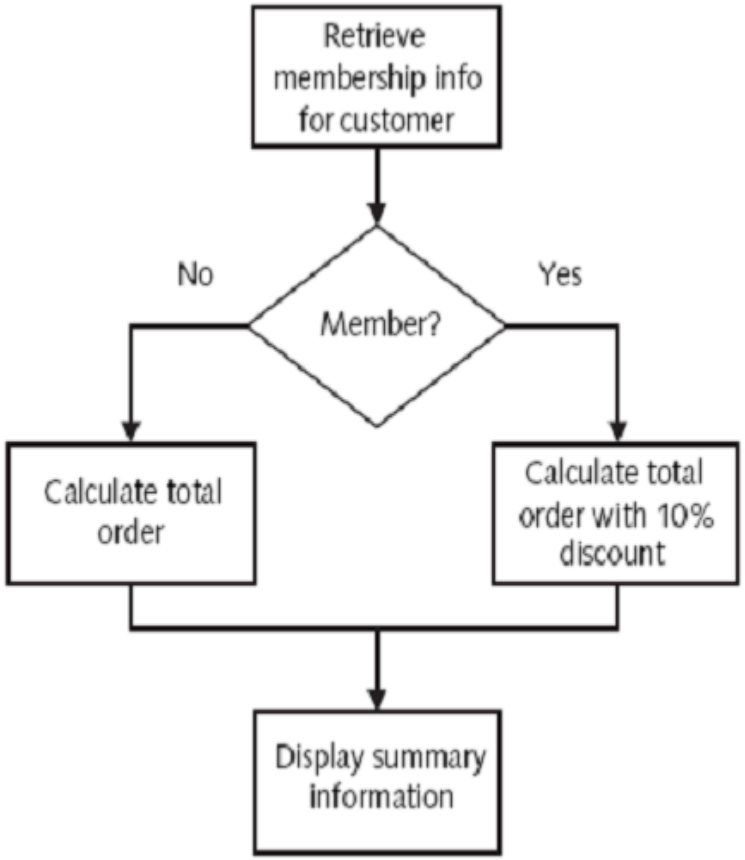
```
1 drop procedure prc_square_num;
```

```
drop procedure prc_square_num succeeded.
```

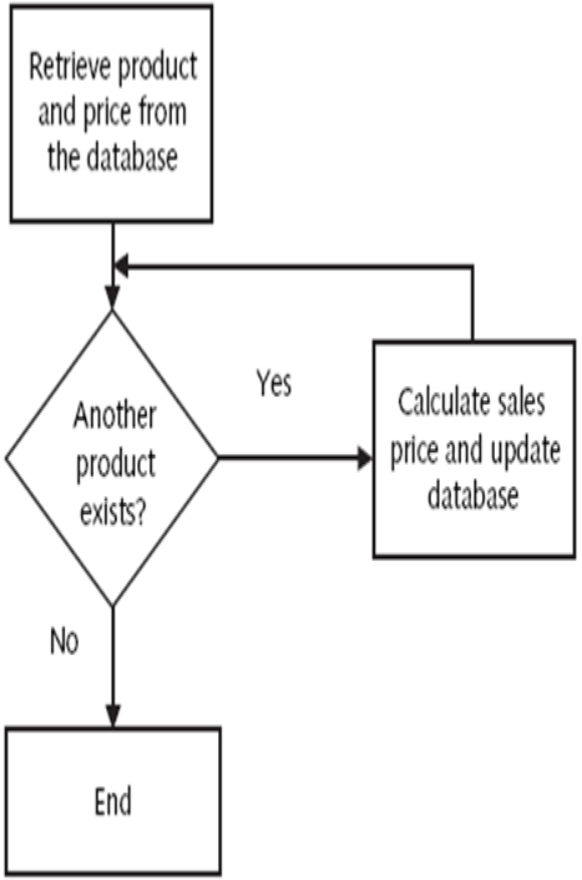
Programming Structures



Flowchart with sequence structure



Flowchart including decision structure



Flowchart including looping structure

Decision or Control Structures

IF Statements

- Simple IF
- IF/THEN/ELSE
- IF/THEN/ELSIF/ELSE

```
IF (condition)  THEN
    statements;
ELSIF (condition)  THEN
 statements;
ELSE
  statements;
END IF;
```

CASE Statements

- Basic CASE statement
- Searched CASE statement
- CASE expression

```
CASE
   WHEN (condition) THEN
     statements;
   WHEN (condition) THEN
     statements;
   ELSE
     statements;
 END CASE;
```

Decision or Control Structures

Given the following example – we will write simple PL/SQL Blocks using

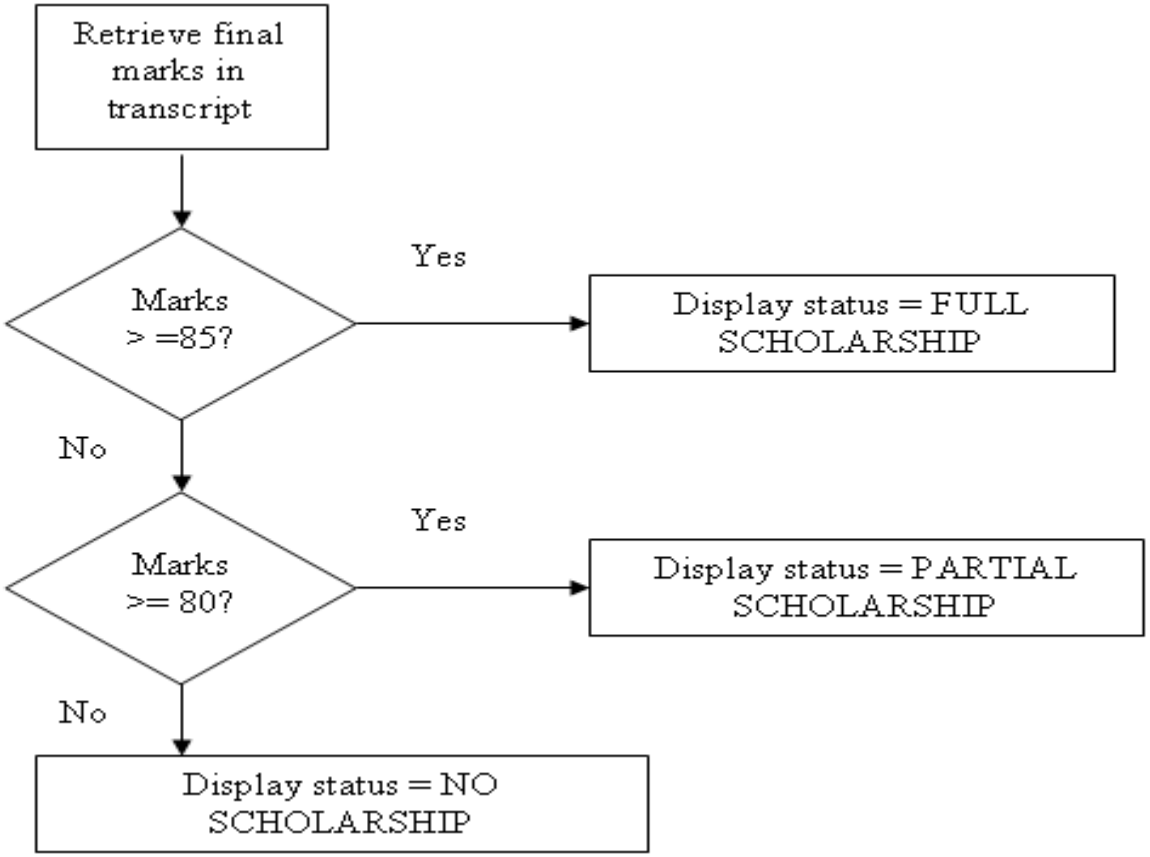
1) If-then else

2) Case Structures

3) Host Variables

4) Named procedures

5) Executing/Dropping procedures



Anonymous PL/SQL Block

```
DECLARE
  v_final_marks NUMBER(2) := 80;
BEGIN
  IF v_final_marks >= 85 THEN
    DBMS_OUTPUT.PUT_LINE('FULL SCHOLARSHIP');
  ELSIF v_final_marks >= 80 THEN
    DBMS_OUTPUT.PUT_LINE('PARTIAL SCHOLARSHIP');
  ELSE
    DBMS_OUTPUT.PUT_LINE('NO SCHOLARSHIP');
  END IF;
END;
```

IF....THEN

ELSIF...THEN

ELSE...

END IF;

CASE

WHEN.....THEN

WHEN.....THEN

ELSE...

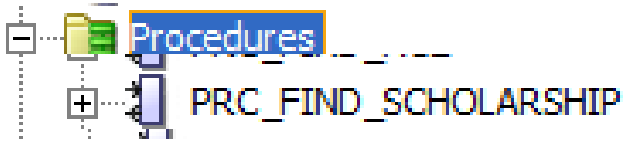
END CASE;

is123456Conn

0.52548677 seconds

```
1  DECLARE
2   v_final_marks NUMBER(2) := 75;
3  BEGIN
4    CASE
5      WHEN v_final_marks  >= 85 THEN
6         DBMS_OUTPUT.PUT_LINE(' FULL SCHOLARSHIP ');
7      WHEN v_final_marks >= 80 THEN
8         DBMS_OUTPUT.PUT_LINE(' PARTIAL SCHOLARSHIP ');
9      ELSE
10        DBMS_OUTPUT.PUT_LINE(' NO SCHOLARSHIP ');
11     END CASE;
12 END;
```

Named PL/SQL Block




```
1  CREATE OR REPLACE PROCEDURE PRC_FIND_SCHOLARSHIP AS
2    v_final_marks NUMBER(2) := 75;
3  BEGIN
4    IF v_final_marks >= 85 THEN
5      DBMS_OUTPUT.PUT_LINE('FULL SCHOLARSHIP');
6    ELSIF v_final_marks >= 80 THEN
7      DBMS_OUTPUT.PUT_LINE('PARTIAL SCHOLARSHIP');
8    ELSE
9      DBMS_OUTPUT.PUT_LINE('NO SCHOLARSHIP');
10   END IF;
11 END;
```

IF....THEN

ELSIF...THEN

ELSE...

END IF;

CASE

WHEN.....THEN

WHEN.....THEN

ELSE...

END CASE;

```
1  CREATE OR REPLACE PROCEDURE PRC_FIND_SCHOLARSHIP AS
2    v_final_marks NUMBER(2) := 75;
3  BEGIN
4    CASE
5      WHEN v_final_marks  >= 85 THEN
6         DBMS_OUTPUT.PUT_LINE(' FULL SCHOLARSHIP ');
7      WHEN v_final_marks >= 80 THEN
8         DBMS_OUTPUT.PUT_LINE(' PARTIAL SCHOLARSHIP ');
9      ELSE
10        DBMS_OUTPUT.PUT_LINE(' NO SCHOLARSHIP ');
11     END CASE;
12 END;
```

SQL* Plus Host Variables

Using application environment variables to send variables into and out of a PL/SQL block

Variables are named memory areas that hold values to allow retrieval and manipulation of values within our programs. These host variables only last for the session

Reference the host variable with a preceding colon in PL/SQL

DATE data type is NOT available for SQL*Plus host variables

Also for numeric host variable – the variable size information is NOT required

Now we will solve the example on the previous slide with host variable and also parameters ... But first what are parameters?

Parameter List

Parameters are the means to pass values to and from the calling environment to the server.

CREATE [OR REPLACE] PROCEDURE <procedure_name> [(parameter_list)] IS

<declarations>

Definition of identifiers e.g. variables, constants, cursors

BEGIN

Here you write statements to manipulate variables in declaration section. This section is mandatory

EXCEPTION

Here you resolve run-time errors

END;

/

Parameter Modes

There are 3 parameter modes – IN, OUT and IN OUT

1. IN – Read Only – passed a value into the program
2. OUT – Write Only – passes a value from the program
3. IN OUT – variable – passes value in and also sends back

The EXECUTE command is used to invoke or call a procedure.

When invoking a procedure

- an argument must be provided for each IN parameter in the procedure.
- A variable (host variable) is declared to hold the value returned via OUT mode

We will re-write the scholarship procedure using IN and OUT variables

Refer slide 22 – Using parameters and host variables

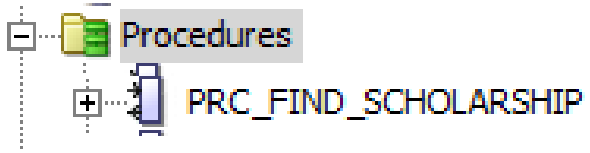
First, we define a host variable called g_result of type VARCHAR2(20)

VARIABLE g_result VARCHAR2(20)

This variable will be used to give the result.

We also need to define a parameter – say, v_final_marks.

This parameter denotes the marks (of type number) which are inputted to the procedure PRC_FIND_SCHOLARSHIP



is123456Conn

0.52441961 seconds

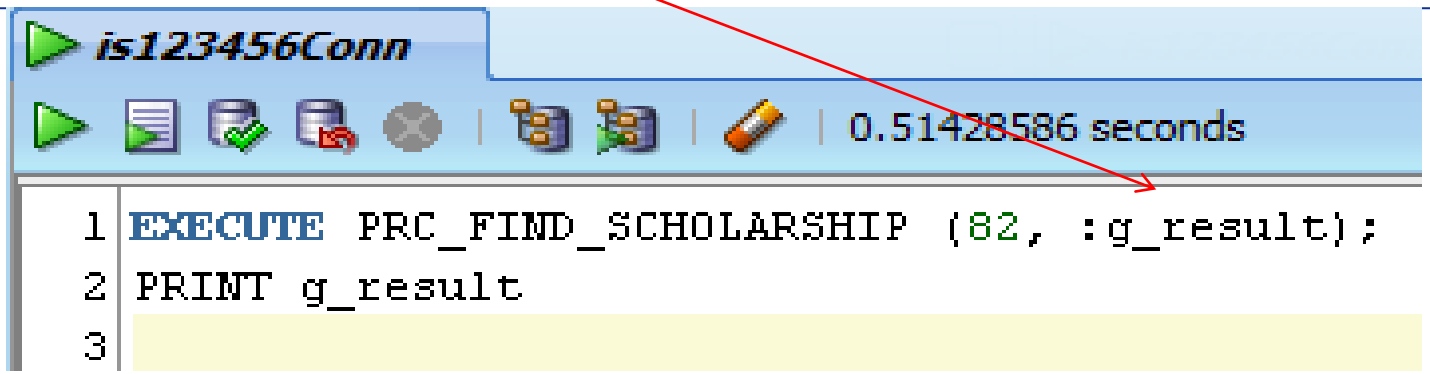
```
1  VARIABLE g_result VARCHAR2(20)
2
3  CREATE OR REPLACE PROCEDURE PRC_FIND_SCHOLARSHIP
4  ( v_final_marks IN NUMBER,
5  v_result OUT VARCHAR2)
6  IS
7  BEGIN
8    IF v_final_marks >= 85 THEN
9    v_result := 'FULL SCHOLARSHIP';
10   ELSIF v_final_marks >= 80 THEN
11       v_result := 'PARTIAL SCHOLARSHIP';
12   ELSE
13        v_result := 'NO SCHOLARSHIP';
14    END IF;
15 END;
```

```
PROCEDURE PRC_FIND_SCHOLARSHIP Compiled.
```

NOTE the host variable g_result and the IN parameter v_final_marks and OUT parameter v_result

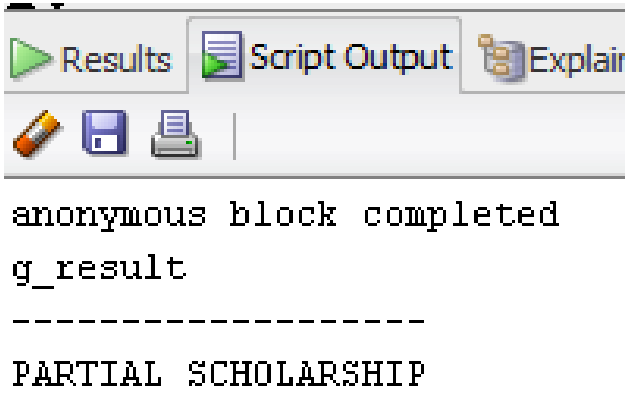
The procedure PRC_FIND_SCHOLARSHIP is now called.

Note the use of g_result to get the value



Thus we can see that the OUT value specified in the parameter list is used by the host variable.

Adding parameters and host variables makes the procedure more flexible



Named PL/SQL Function

Use CREATE OR REPLACE FUNCTION command

function_return_value_datatype - Defines data type that function returns

End with the RETURN command

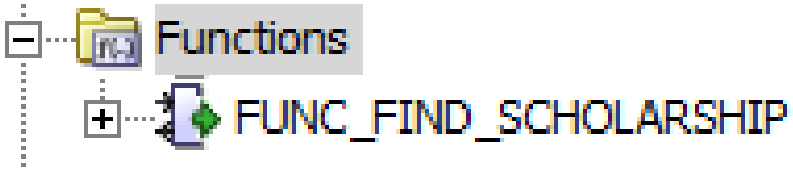
A function is called by assigning the return value of the function to a variable.

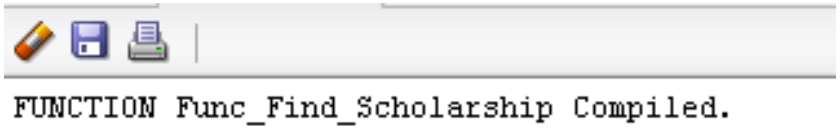
variable_name := *function_name(parameter1, parameter2,* ...);

```
CREATE OR REPLACE FUNCTION FUNCTION_NAME
    (parameter list to be sent in)
RETURN datatype
IS
    function_return_variable_name datatype;
BEGIN
   -- executable statements;
    RETURN          function_return_variable_name
END;
/
DROP FUNCTION FUNCTION_NAME
```

Named PL/SQL Function
Refer slide 22

```
1  CREATE OR REPLACE
2    FUNCTION Func_Find_Scholarship
3       (
4         v_final_marks NUMBER)
5      RETURN VARCHAR2
6    IS
7      v_result VARCHAR2(20);
8    BEGIN
9      IF v_final_marks    >= 85 THEN
10       v_result          := 'FULL SCHOLARSHIP';
11     ELSIF v_final_marks >= 80 THEN
12       v_result          := 'PARTIAL SCHOLARSHIP';
13     ELSE
14       v_result := 'NO SCHOLARSHIP';
15     END IF;
16     RETURN v_result;
17   END;
```

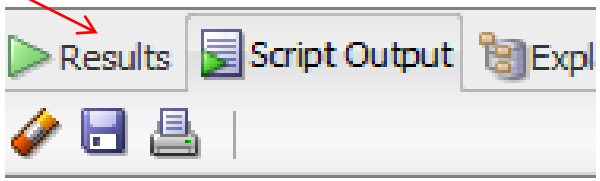




Named PL/SQL Function

Now call this function thru a procedure

```
BEGIN
  dbms_output.put_line(Func_Find_Scholarship(85));
END;
```




```
anonymous block completed
FULL SCHOLARSHIP
```

So, procedures and functions both can be used – though their method of calling them is different.

Loop Structures

- Enables a statement or set of statements to be executed more than once
- A loop must provide instructions of when to end the looping, or an 'infinite' loop will be produced.
- Three forms of loops exist: Basic, While and For. The main difference of the forms is how the loop is ended.

```
BEGIN
    LOOP
      executable statements;
      EXIT WHEN condition;
      increment/decrement statements;
    END LOOP;
END;
```

OR

```
BEGIN
    WHILE condition LOOP
      executable statements;
      increment/decrement statements;
    END LOOP;
END;
```

OR

```
BEGIN
    FOR variable in start..end LOOP
          executable statements;
    END LOOP;
END;
```

Loops

```
1 DECLARE
2 v_count_num NUMBER(2) := 1;
3 BEGIN
4 LOOP
5 DBMS_OUTPUT.PUT_LINE(v_count_num);
6 EXIT WHEN v_count_num >= 5;
7 v_count_num := v_count_num + 1;
8 END LOOP;
9 END;
```

LOOP - When

```
1 DECLARE
2 v_count_num NUMBER(2) := 1;
3 BEGIN
4 WHILE v_count_num <= 5 LOOP
5 DBMS_OUTPUT.PUT_LINE(v_count_num);
6 v_count_num := v_count_num + 1;
7 END LOOP;
8 END;
```

LOOP - While

```
1 DECLARE
2 v_count_num NUMBER(2) := 1;
3 BEGIN
4 FOR v_count_num in 1..5 LOOP
5 DBMS_OUTPUT.PUT_LINE(v_count_num);
6 END LOOP;
7 END;
```

LOOP - For

SQL Queries within PL/SQL

SQL queries can be embedded into a PL/SQL block to retrieve data.

The SQL statement differs slightly – i.e. the INTO clause is needed that instructs the program into which variables to place the values returned from the database.

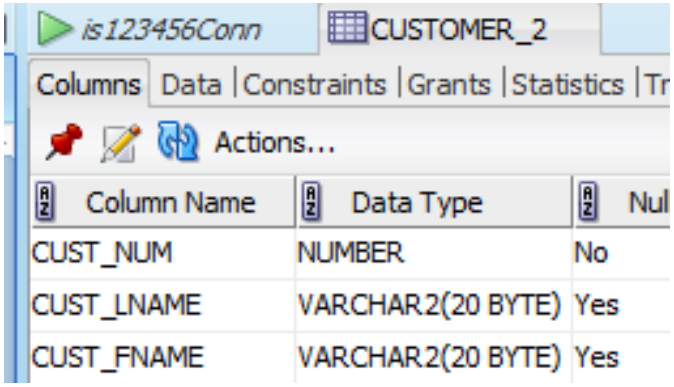
If scalar variables (CHAR, VARCHAR, NUMBER, DATE, BOOLEAN) are used, the query can only return one row and a variable must exist for each column value returned.

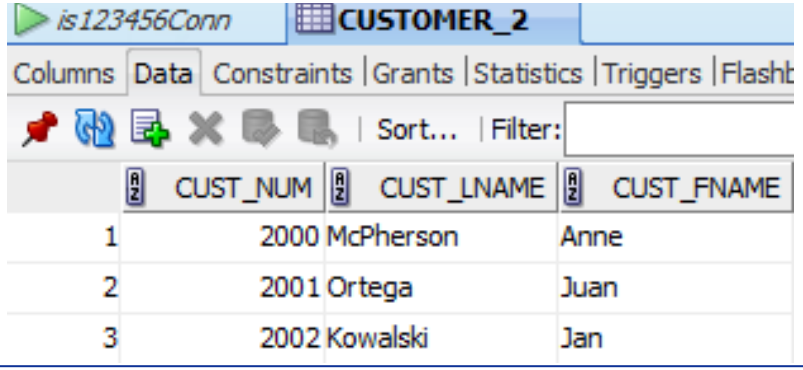
SQL Queries with Scalar Type

Let us create a table CUSTOMER_2 as follows

```
CREATE TABLE CUSTOMER_2 (
CUST_NUM      NUMBER PRIMARY KEY,
CUST_LNAME    VARCHAR(20),
CUST_FNAME    VARCHAR(20));

INSERT INTO CUSTOMER_2 VALUES(2000 ,'McPherson' ,'Anne');
INSERT INTO CUSTOMER_2 VALUES(2001 ,'Ortega'    ,'Juan');
INSERT INTO CUSTOMER_2 VALUES(2002 ,'Kowalski'  ,'Jan');
```





SQL Queries with Scalar Type

SQL statement as used in PL/SQL

```
SELECT  cust_fname, cust_lname
INTO v_cust_name, v_cust_name
FROM customer_2
WHERE cust_num = 2000;
```

is123456Conn | CUSTOMER_2

Columns | Data | Constraints | Grants | Statistics | Triggers | Flashb

Sort... | Filter:

	CUST_NUM	CUST_LNAME	CUST_FNAME
1	2000	McPherson	Anne
2	2001	Ortega	Juan
3	2002	Kowalski	Jan

```
1  DECLARE
2    v_cust_num   NUMBER;
3    v_cust_lname VARCHAR2(20);
4    v_cust_fname VARCHAR2(20);
5  BEGIN
6    SELECT cust_num,
7      cust_fname,
8      cust_lname
9    INTO v_cust_num,
10     v_cust_fname,
11     v_cust_lname
12   FROM customer_2
13   WHERE cust_num = 2000;
14
15   dbms_output.put_line(' Customer details are: customer number '
16     || v_cust_num || ' , customer name ' || v_cust_fname || ' '
17     || v_cust_lname);
18 END;
```

SELECT cust_fname, cust_lname

INTO v_cust_name, v_cust_name

FROM customer_2

WHERE cust_num = 2000;

PL/SQL with DML statements

INSERTstatement can be embedded in PL/SQL blocks

```
BEGIN
INSERT INTO CUSTOMER_2 VALUES(2003 ,'Chen','George');
COMMIT;
END;
```

```
1 BEGIN
2 INSERT INTO CUSTOMER_2 VALUES(2003 ,'Chen','George');
3 COMMIT;
4 END;
```

Sort... | Filter:

	CUST_NUM	CUST_LNAME	CUST_FNAME
1	2000	McPherson	Anne
2	2001	Ortega	Juan
3	2002	Kowalski	Jan
4	2003	Chen	George

PL/SQL with DML statements

UPDATE statement can be embedded in PL/SQL blocks

```
BEGIN
UPDATE CUSTOMER_2
SET CUST_LNAME = 'ZZZZZZZZ'
WHERECUST_NUM = 2002;
COMMIT;
END;
```

```
1 BEGIN
2 UPDATE CUSTOMER_2
3 SET CUST_LNAME = 'ZZZZZZZZ'
4 WHERE CUST_NUM = 2002;
5 COMMIT;
6 END;
```

	CUST_NUM	CUST_LNAME	CUST_FNAME
1	2000	McPherson	Anne
2	2001	Ortega	Juan
3	2002	ZZZZZZZZ	Jan
4	2003	Chen	George

PL/SQL with DML statements

DELETE statement can be embedded in PL/SQL blocks

```
BEGIN
DELETE FROM CUSTOMER_2
WHERE CUST_NUM = 2002;
COMMIT;
END;
```

```
1 BEGIN
2 DELETE FROM CUSTOMER_2
3 WHERE CUST_NUM = 2002;
4 COMMIT;
5 END;
```

CUSTOMER_2

Columns | Data | Constraints | Grants | Statistics | Triggers | Flashb

Sort... | Filter:

	CUST_NUM	CUST_LNAME	CUST_FNAME
1	2000	McPherson	Anne
2	2001	Ortega	Juan
3	2003	Chen	George

SQL Queries within %Type

The %TYPE Attribute or anchored data type is used in variable declaration to provide data type based on a table column.

Advantages - reduces code maintenance since database structure changes are automatically reflected in the PL/SQL variable

Disadvantages - additional processing is required to retrieve the data type

v_variablename *tablename.columnname*%TYPE

Creates a record structure based on table structure

SQL Queries within %Type

is123456Conn | CUSTOMER_2

0.03382338 seconds

```
1  DECLARE
2  v_cust_num customer_2.cust_num%TYPE;
3  v_cust_lname customer_2. cust_lname%TYPE;
4  v_cust_fname customer_2.cust_fname%TYPE;
5  BEGIN
6  select cust_num, cust_fname, cust_lname
7  into v_cust_num, v_cust_fname, v_cust_fname
8  from customer_2
9  where cust_num = 2000;
10
11 dbms_output.put_line('
12   Customer details are: customer number ' || v_cust_num || ' ,
13   customer name ' || v_cust_fname ||  ' ' ||  v_cust_lname);
14 END;
```

```
Customer details are: customer number 2000 ,
customer name McPherson
```

Exception Handling

Used to capture error conditions and handle the processing to allow the application to continue.

Placed in the EXCEPTION section of a PL/SQL block.

There are two types of exceptions:

1. **Predefined exceptions** -- exception names associated with common Oracle errors
2. **User-defined exception** -- an exception that a developer explicitly raises in the block to enforce a business rule

Pre-defined User Exceptions

NO_DATA_FOUND - A SELECT statement in a PL/SQL block retrieves no rows or a nonexistent row of an index-by table is referenced

TOO_MANY_ROWS - A SELECT statement in a PL/SQL block retrieves more than one row

CASE_NOT_FOUND - No WHEN clause in the CASE statement is processed

ZERO_DIVIDE - Attempted division by zero

DUP_VAL_ON_INDEX - Attempted violation of a unique or primary key column constraint

NO_DATA_FOUND Exception

```
 1 DECLARE
 2 v_cust_num customer_2.cust_num%TYPE;
 3 v_cust_lname customer_2. cust_lname%TYPE;
 4 v_cust_fname customer_2.cust_fname%TYPE;
 5 BEGIN
 6 select cust_num, cust_fname, cust_lname
 7 into v_cust_num, v_cust_fname, v_cust_fname
 8 from customer_2
 9 where cust_num = 2005;
10
11 dbms_output.put_line('
12   Customer details are: customer number '
13   || v_cust_num || ' ,
14     customer name '
15   || v_cust_fname ||  ' ' ||  v_cust_lname);
16
17 EXCEPTION
18 when no_data_found then
19 dbms_output.put_line ('There is no such customer');
20 END;
```

TOO_MANY_ROWS Exception

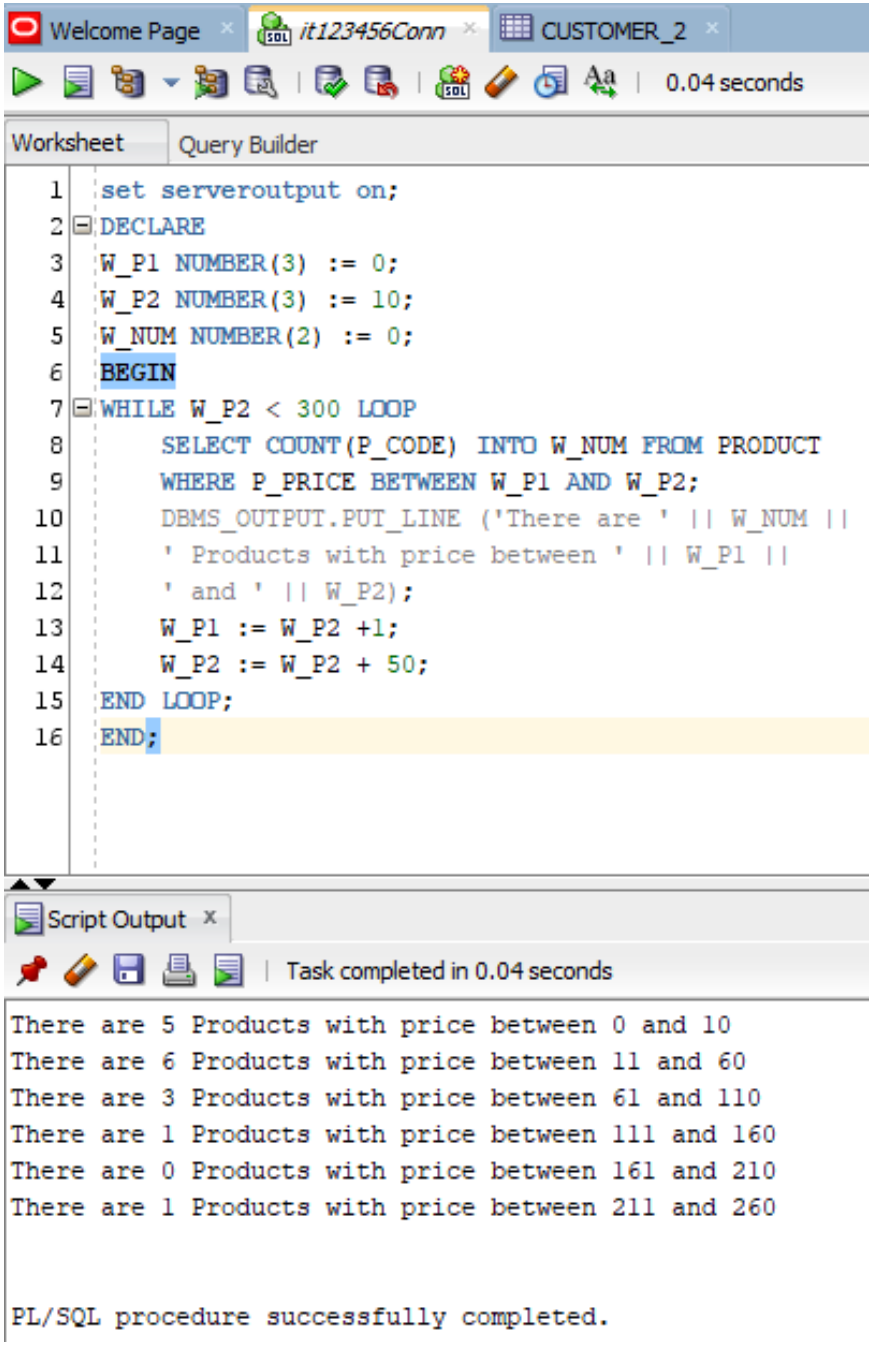
	STUD_NUM	STUD_LNAME	STUD_FNAME	STUD_AGE
1	2001	McPherson	Anne	24
2	2002	Ortega	Juan	20
3	2003	Kowalski	Jan	18
4	2004	Smith	Jane	38
5	2005	Kane	Sam	48

```
This student is a teenager Jan Kowalski . Age is 18
```

```
set serveroutput on;
DECLARE
    v_student_num     student.stud_num%TYPE;
    v_student_fname   student.stud_fname%TYPE;
    v_student_lname   student.stud_lname%TYPE;
    v_student_age     student.stud_age%TYPE;
    begin SELECT
    stud_num,
    stud_fname,
    stud_lname,
    stud_age
INTO
    v_student_num,v_student_fname,v_student_lname,v_student_age
     FROM
    student
     WHERE
    stud_age < 19; dbms_output.put_line ( 'This student is a teenager '
|| v_student_fname
|| ' '
|| v_student_lname
|| ' '
|| '. Age is ' || v_student_age);
EXCEPTION
    WHEN no_data_found THEN
        dbms_output.put_line('No such student');
    WHEN too_many_rows THEN
        dbms_output.put_line('There are many students');
      end;
```

Book examples

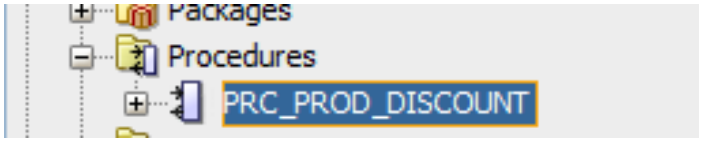
```
DECLARE
W_P1 NUMBER(3) := 0;
W_P2 NUMBER(3) := 10;
W_NUM NUMBER(2) := 0;
BEGIN
WHILE W_P2 < 300 LOOP
SELECT COUNT(P_CODE) INTO W_NUM FROM PRODUCT
WHERE P_PRICE BETWEEN W_P1 AND W_P2;
DBMS_OUTPUT.PUT_LINE ('There are ' || W_NUM ||
' Products with price between ' || W_P1 || ' and ' || W_P2);
W_P1 := W_P2 +1;
W_P2 := W_P2 + 50;
END LOOP;
END;
```



Book examples

```
SELECT P_CODE, P_DISCOUNT FROM PRODUCT;


CREATE OR REPLACE PROCEDURE
      PRC_PROD_DISCOUNT
AS BEGIN
UPDATE PRODUCT
   SET P_DISCOUNT = P_DISCOUNT + .05
   WHERE P_QOH >= P_MIN *2;
DBMS_OUTPUT.PUT_LINE ('** Update Finished **');
END;
/
```




```
BEGIN
PRC_PROD_DISCOUNT;
END;

SELECT P_CODE, P_DISCOUNT FROM PRODUCT;
```

Script Output | Query Result

SQL | All Rows Fet

	P_CODE	P_DISCOUNT
1	11QER/31	0
2	13-Q2/P2	0.05
3	14-Q1/L3	0
4	1546-QQ2	0
5	1558-QW1	0
6	2232/QTY	0.05
7	2232/QWE	0.05
8	2238/QPD	0.05
9	23109-HB	0.1
10	23114-AA	0.05
11	54778-2T	0
12	89-WRE-Q	0.05
13	PVC23DRT	0
14	SM-18277	0
15	SW-23116	0

Script Output | Query Result

SQL | All Rows Fetch

	P_CODE	P_DISCOUNT
1	11QER/31	0
2	13-Q2/P2	0.1
3	14-Q1/L3	0
4	1546-QQ2	0
5	1558-QW1	0.05
6	2232/QTY	0.05
7	2232/QWE	0.05
8	2238/QPD	0.1
9	23109-HB	0.15
10	23114-AA	0.05
11	54778-2T	0.05
12	89-WRE-Q	0.1
13	PVC23DRT	0.05
14	SM-18277	0.05
15	SW-23116	0.05
16	WR3/TT3	0.15

PL/SQL

As you can see, PL/SQL is Oracle's procedural extension to SQL.

PL/SQL is not a stand-alone programming language, but exists only as a tool within the Oracle programming environment.

It has program structure constructs, such as procedure blocks and functions using programming logic -conditional statements, loops, customized exception handling and structured data.

It is strongly typed language, i.e. each variable needs to be explicitly declared with its specific data type before it can be used.